Отчет

Этап 1

создаем каталог, выставляем права и проверяем, что каталог создался

```
export PGDATA=$HOME/kqz19
mkdir -p "$PGDATA"
chmod 700 "$PGDATA"
ls -ld "$PGDATA"
```

ищем все русские локали:

```
locale -a | grep 'ru_RU'
ru_RU.CP1251
ru_RU.CP866
ru_RU.ISO8859-5
ru_RU.KOI8-R
ru_RU.UTF-8
```

нам нужна `ru_RU.CP1251`

В Postgres кодировка `ANSI1251` называется `WIN1251`:

создание кластера

```
initdb -D "$PGDATA" \
  --encoding=WIN1251 \
  --locale=ru_RU.CP1251
```

проверим, что локаль выставлена корректно:

```
grep -E "^(# )?(lc_|server_encoding)" "$PGDATA/postgresql.conf" | sed
's/^# //'
lc_messages = 'ru_RU.CP1251'            # locale for system error message
lc_monetary = 'ru_RU.CP1251'            # locale for monetary formatting
lc_numeric = 'ru_RU.CP1251'             # locale for number formatting
lc_time = 'ru_RU.CP1251'                # locale for time formatting
```

старт

```
pg_ctl -D $PGDATA -l $PGDATA/server.log start
```

Этап 2

pg_hba.conf

- Способы подключения:
    1. Unix-domain сокет в режиме peer;
    2. сокет TCP/IP, принимать подключения к любому IP-адресу узла
- Способ аутентификации TCP/IP клиентов: по имени пользователя
- Остальные способы подключений запретить.

Способ аутентификации TCP/IP клиентов по имени пользователя в PostgreSQL это ident

```
cp "$PGDATA/pg_hba.conf" "$PGDATA/pg_hba.conf.bak_stage2"

cat > "$PGDATA/pg_hba.conf" <<'EOF'
# TYPE  DATABASE  USER  ADDRESS         METHOD

# 1) Unix-domain сокет: peer
local   all       all                   peer

# 2) TCP/IP к любому IP-адресу узла: ident (по имени пользователя)
host    all       all   0.0.0.0/0       ident
host    all       all   ::/0            ident
EOF
```

проверка:

```
cat "$PGDATA/pg_hba.conf"
```

postgresql.conf

делаем бэкап `postgresql.conf` и создаем каталог для логов

```
cp "$PGDATA/postgresql.conf" "$PGDATA/postgresql.conf.bak_stage2"
mkdir -p "$PGDATA/log"
chmod 700 "$PGDATA/log"
```

- Номер порта: `9041`
- Настроить следующие параметры сервера БД:
    - max_connections

    - shared_buffers
    - temp_buffers
    - work_mem
    - checkpoint_timeout
    - effective_cache_size
    - fsync
    - commit_delay

      Параметры должны быть подобраны в соответствии со сценарием OLTP: 500 транзакций в секунду размером 8КБ; обеспечить высокую доступность (High Availability) данных

- Формат лог-файлов: `.log`
- Уровень сообщений лога: `WARNING`
- Дополнительно логировать: попытки подключения и завершение сессий

```
cat >> "$PGDATA/postgresql.conf" <<'EOF'

# === Task 2 settings ===
# Connections
listen_addresses = '*'
port = 9041

# Logging
logging_collector = on
log_directory = 'log'
log_filename = 'server.log'          # формат .log
log_min_messages = warning
log_connections = on
log_disconnections = on
log_destination = 'stderr'

# OLTP-ish tuning (safe defaults without knowing RAM)
max_connections = 200
shared_buffers = 256MB
temp_buffers = 16MB
work_mem = 4MB
checkpoint_timeout = 15min
effective_cache_size = 768MB
fsync = on
commit_delay = 100
EOF
```

проверка, что строки добавились:

```
tail -n 30 "$PGDATA/postgresql.conf"
```

- Директория WAL файлов: `$HOME/zto30`

В PG16 `WAL` лежит в `PGDATA/pg_wal`. Вынесем его в отдельный каталог и сделаем симлинк.

```
mkdir -p "$HOME/zto30"
chmod 700 "$HOME/zto30"

# перенос WAL и создание симлинка
mv "$PGDATA/pg_wal" "$HOME/zto30/pg_wal"
ln -s "$HOME/zto30/pg_wal" "$PGDATA/pg_wal"

# проверка
ls -la "$PGDATA" | egrep "pg_wal"
lrwxr-xr-x   1 postgres1 postgres    30  1 марта 16:51 pg_wal ->
/var/db/postgres1/zto30/pg_wal

ls -ld "$HOME/zto30/pg_wal"
drwx------  3 postgres1 postgres 4  1 марта 15:36
/var/db/postgres1/zto30/pg_wal
```

Запуск сервера и проверка

```
pg_ctl -D "$PGDATA" -l "$PGDATA/pg_ctl.log" start
ожидание запуска сервера.... готово
сервер запущен

pg_ctl -D "$PGDATA" status
pg_ctl: сервер работает (PID: 39541)
/usr/local/bin/postgres "-D" "/var/db/postgres1/kqz19"
```

Проверим, что сокет TCP/IP слушает порт 9041 и что настройки логгера применились:

```
psql -p 9041 -d postgres -c "show port; show listen_addresses; show
server_encoding;"
 port
------
 9041
(1 строка)

 listen_addresses
------------------
 *
(1 строка)
```

```
 server_encoding
-----------------
 WIN1251
(1 строка)

psql -p 9041 -d postgres -c "show log_min_messages; show log_destination;
show logging_collector; show log_connections; show log_disconnections;"
 log_min_messages
------------------
 warning
(1 строка)

 log_destination
-----------------
 stderr
(1 строка)

 logging_collector
-------------------
 on
(1 строка)

 log_connections
-----------------
 on
(1 строка)

 log_disconnections
--------------------
 on
(1 строка)
```

Проверим локаль и кодировку:

```
psql -p 9041 -d postgres -c " SELECT current_database() AS db,
pg_encoding_to_char(encoding) AS encoding, datcollate, datctype FROM
pg_database WHERE datname = current_database(); "

    db    | encoding |  datcollate  |   datctype
----------+----------+--------------+--------------
 postgres | WIN1251  | ru_RU.CP1251 | ru_RU.CP1251
(1 строка)
```

Unix-peer сокет работает:

```
psql -p 9041 -d postgres -c "select inet_client_addr(),
inet_client_port();"
 inet_client_addr | inet_client_port
```

```
-----------------+-----------------
                 |
(1 строка)
```

Убедимся, что лог действительно .log и пишется:

```
ls -la "$PGDATA/log"

tail -n 50 "$PGDATA/log/server.log"

2026-03-01 17:27:49.756 MSK [47983] СООБЩЕНИЕ:  запускается PostgreSQL
16.4 on amd64-portbld-freebsd14.1, compiled by FreeBSD clang version
18.1.6 (https://github.com/llvm/llvm-project.git llvmorg-18.1.6-0-
g1118c2e05e67), 64-bit
2026-03-01 17:27:49.756 MSK [47983] СООБЩЕНИЕ:  для приёма подключений по
адресу IPv6 "::" открыт порт 9041
2026-03-01 17:27:49.756 MSK [47983] СООБЩЕНИЕ:  для приёма подключений по
адресу IPv4 "0.0.0.0" открыт порт 9041
2026-03-01 17:27:49.765 MSK [47983] СООБЩЕНИЕ:  для приёма подключений
открыт Unix-сокет "/tmp/.s.PGSQL.9041"
2026-03-01 17:27:49.792 MSK [47987] СООБЩЕНИЕ:  система БД была выключена:
2026-03-01 17:22:20 MSK
2026-03-01 17:27:49.807 MSK [47983] СООБЩЕНИЕ:  система БД готова
принимать подключения
```

Этап 3

- Создать новые табличные пространства для различных таблиц: `$HOME/ndx81`, `$HOME/beb52`, `$HOME/smq7`

создаем каталоги и выдаем права

```
mkdir -p "$HOME/ndx81" "$HOME/beb52" "$HOME/smq7"
chmod 700 "$HOME/ndx81" "$HOME/beb52" "$HOME/smq7"
ls -ld "$HOME/ndx81" "$HOME/beb52" "$HOME/smq7"
```

создаем табличные пространства

```
psql -p 9041 -d postgres -v ON_ERROR_STOP=1 <<'SQL'
CREATE TABLESPACE ts_ndx81 LOCATION '/var/db/postgres1/ndx81';
CREATE TABLESPACE ts_beb52 LOCATION '/var/db/postgres1/beb52';
CREATE TABLESPACE ts_smq7  LOCATION '/var/db/postgres1/smq7';
SQL
```

проверка:

```
[postgres1@pg113 ~]$ psql -p 9041 -d postgres -c "\db+"
                                    Список табличных пространств
    Имя     | Владелец  |      Расположение       | Права доступа |
Параметры | Размер  | Описание
------------+-----------+-------------------------+---------------+-------
----+---------+----------
 pg_default | postgres1 |                         |               |
| 22 MB   |
 pg_global  | postgres1 |                         |               |
| 565 kB  |
 ts_beb52   | postgres1 | /var/db/postgres1/beb52 |               |
| 0 bytes |
 ts_ndx81   | postgres1 | /var/db/postgres1/ndx81 |               |
| 0 bytes |
 ts_smq7    | postgres1 | /var/db/postgres1/smq7  |               |
| 0 bytes |
(5 строк)
```

- На основе `template1` создать новую базу: `fakebrownwood`

Используем `ts_ndx81` как `default` для базы

```
psql -p 9041 -d postgres -v ON_ERROR_STOP=1 <<'SQL'
CREATE DATABASE fakebrownwood
  WITH TEMPLATE template1
       OWNER postgres1
       TABLESPACE ts_ndx81;
SQL
```

проверка:

```
psql -p 9041 -d postgres -c "\l+ fakebrownwood"

Список баз данных
      Имя      | Владелец  | Кодировка | Провайдер локали |  LC_COLLATE  |
LC_CTYPE   | локаль ICU | Правила ICU | Права доступа | Размер  | Табл.
пространство | Описание
---------------+-----------+-----------+------------------+--------------
+--------------+------------+-------------+---------------+---------+-----
---------------+----------
 fakebrownwood | postgres1 | WIN1251   | libc             | ru_RU.CP1251 |
ru_RU.CP1251 |            |             |               | 7513 kB |
ts_ndx81           |
(1 строка)
```

- Создать новую роль, предоставить необходимые права, разрешить подключение к базе.

Создаём роль с паролем и правом логина

```
psql -p 9041 -d postgres -v ON_ERROR_STOP=1 <<'SQL'
CREATE ROLE role_lab
  LOGIN
  PASSWORD 'ChangeMe_123';
SQL
```

Разрешаем подключение к базе и даём базовые права

```
psql -p 9041 -d postgres -v ON_ERROR_STOP=1 <<'SQL'
GRANT CONNECT ON DATABASE fakebrownwood TO role_lab;
GRANT CREATE ON DATABASE fakebrownwood TO role_lab;
GRANT CREATE ON TABLESPACE ts_ndx81 TO role_lab;
GRANT CREATE ON TABLESPACE ts_beb52 TO role_lab;
GRANT CREATE ON TABLESPACE ts_smq7  TO role_lab;
SQL
```

- От имени новой роли (не администратора) произвести наполнение ВСЕХ созданных баз тестовыми наборами данных. ВСЕ табличные пространства должны использоваться по назначению.

```
Таблицы данных → ts_ndx81 (у базы он как default)

Индексы → ts_beb52

Большую таблицу/архив (или отдельную таблицу) → ts_smq7
```

Режим peer означает: PostgreSQL берёт имя пользователя ОС (postgres1) и пытается зайти в БД под ролью с таким же именем.

зайти как role_lab не получится, тк ОС-пользователь всё равно postgres1 → поэтому подредактируем `pg_hba.conf`:

```
# TYPE  DATABASE        USER        ADDRESS         METHOD

# разрешаем role_lab подключаться по паролю через Unix socket
local   fakebrownwood   role_lab                    scram-sha-256
```

Перезагружаем сервер

```
pg_ctl -D "$PGDATA" reload
```

Подключаемся под админом и выдаем права role_lab

```
psql -p 9041 -d fakebrownwood
```

```
GRANT USAGE, CREATE ON SCHEMA public TO role_lab;
```

Подключаемся под новой ролью с паролем:

```
psql -p 9041 -U role_lab -d fakebrownwood
```

- Таблица пользователей

  не укзаываем тейблспейс явно, тк она попадет в дефолтный (`ts_ndx81`)

  ```
  CREATE TABLE users (
    id bigserial PRIMARY KEY,
    username varchar(64) NOT NULL UNIQUE,
    created_at timestamp DEFAULT now()
  );
  ```

- Таблица заказов в другом tablespace `ts_smq7`

  ```
  CREATE TABLE orders (
    id bigserial PRIMARY KEY,
    user_id bigint REFERENCES users(id),
    amount integer,
    created_at timestamp DEFAULT now()
  ) TABLESPACE ts_smq7;
  ```

- Индекс в третьем tablespace `ts_beb52`

  ```
  CREATE INDEX orders_user_idx
  ON orders(user_id)
  TABLESPACE ts_beb52;
  ```

заполняем тестовыми данными:

```
INSERT INTO users(username)
SELECT 'user_' || g
FROM generate_series(1,50) g;
```

```
INSERT INTO orders(user_id, amount)
SELECT (random()*49)::int + 1,
       (random()*10000)::int
FROM generate_series(1,500);
```

проверка на вставку в бд:

```
SELECT
 (SELECT count(*) FROM users)  AS users_cnt,
 (SELECT count(*) FROM orders) AS orders_cnt;

  users_cnt | orders_cnt
 -----------+------------
         50 |        500
(1 строка)
```

Проверка, что все табличные пространства используются

```
SELECT
  c.relname AS object,
  c.relkind AS type,
  COALESCE(t.spcname, 'pg_default') AS tablespace
FROM pg_class c
LEFT JOIN pg_tablespace t ON t.oid = c.reltablespace
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE n.nspname = 'public'
  AND c.relname IN ('users','orders','orders_user_idx')
ORDER BY type, object;

     object      | type | tablespace
-----------------+------+------------
 orders_user_idx | i    | ts_beb52
 orders          | r    | ts_smq7
 users           | r    | pg_default
(3 строки)
```

вывести все объекты и их tablespace:

```
SELECT
  COALESCE(t.spcname, 'pg_default') AS tablespace,
  n.nspname AS schema,
  c.relname AS object,
  CASE c.relkind
    WHEN 'r' THEN 'table'
    WHEN 'i' THEN 'index'
```

```
    WHEN 'm' THEN 'matview'
    WHEN 'S' THEN 'sequence'
    ELSE c.relkind::text
  END AS type
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
LEFT JOIN pg_tablespace t ON t.oid = c.reltablespace
WHERE n.nspname = 'public'
  AND c.relkind IN ('r','i','S')
ORDER BY 1, 2, 4, 3;

 tablespace | schema |       object       |   type
------------+--------+--------------------+----------
 pg_default | public | orders_pkey        | index
 pg_default | public | users_pkey         | index
 pg_default | public | users_username_key | index
 pg_default | public | orders_id_seq      | sequence
 pg_default | public | users_id_seq       | sequence
 pg_default | public | users              | table
 ts_beb52   | public | orders_user_idx    | index
 ts_smq7    | public | orders             | table
(8 строк)
```

доказательство, что pg_default это ts_ndx81

```
SELECT
  d.datname,
  COALESCE(t.spcname, 'pg_default') AS default_tablespace
FROM pg_database d
LEFT JOIN pg_tablespace t ON t.oid = d.dattablespace
WHERE d.datname = 'fakebrownwood';

    datname    | default_tablespace
---------------+--------------------
 fakebrownwood | ts_ndx81
(1 строка)
```

Список всех tablespace

```
postgres=# \db+
                                        Список табличных пространств
    Имя     | Владелец  |      Расположение       |     Права доступа
| Параметры | Размер  | Описание
------------+-----------+-------------------------+-----------------------
+-----------+---------+----------
 pg_default | postgres1 |                         |
|           | 22 MB   |
 pg_global  | postgres1 |                         |
```

```
|           | 589 kB  |
 ts_beb52   | postgres1 | /var/db/postgres1/beb52 |
postgres1=C/postgres1+|           | 16 kB   |
            |           |                         | role_lab=C/postgres1
|           |         |
 ts_ndx81   | postgres1 | /var/db/postgres1/ndx81 |
postgres1=C/postgres1+|           | 7845 kB |
            |           |                         | role_lab=C/postgres1
|           |         |
 ts_smq7    | postgres1 | /var/db/postgres1/smq7  |
postgres1=C/postgres1+|           | 56 kB   |
            |           |                         | role_lab=C/postgres1
|           |         |
(5 строк)
```